AI 디지털 글쓰기 강의 스크립트

기준 자료: `docs/presentation.md`

전체 시간

구간	슬라이드	시간
도입과 문서 조건	1-5	15분
DocuMeta 전후 비교	6-9	25분
RAG 개념 정리	10-13	20분
DocuMeta 프레임 생성	14-16	25분
RAG와 출력 빌드	17-18	25분
제품화와 정리	19-22	10분
합계	22장	120분

1. 문장과 문서 사이의 마법

사용 시간: 3분

설명: 오늘은 AI가 글을 잘 쓰는지 보는 시간이 아니라, 조직 문서가 시스템 안에서 어떻게 만들어지는지 보는 시간이다. 최종 결과만 보지 않고 문서 조건, 근거 검색, 검증, 출력까지 이어지는 길을 따라간다.

전환: 먼저 업무 문서는 왜 문장만으로 부족한지 본다.

2. 1부: 문서의 규칙과 숨은 질서

사용 시간: 2분

설명: 1부에서는 문장보다 앞에 있는 문서의 규칙을 잡는다. 문서 조건을 태깅하고, 그것을 DocuMeta로 고정한 뒤, RAG가 어디에 붙는지까지 연결한다.

전환: GPT가 문장을 잘 쓰는 것과 조직 문서를 안정적으로 만드는 것은 다르다.

3. 잘 쓴 문장과 아직 부족한 문서

사용 시간: 4분

설명: GPT는 이미 문장을 자연스럽게 쓴다. 하지만 조직 문서에는 독자, 근거, 필수 항목, 최종 출력 형식이 함께 필요하다. 이 네 가지가 분리되어 있지 않으면 결과가 그럴듯해도 검토와 재사용이 어렵다.

전환: 그래서 문서를 문장력이 아니라 규칙의 묶음으로 본다.

4. 문서를 이루는 네 가지 규칙

사용 시간: 4분

설명: 업무 문서는 대상, 형식, 근거, 출력 네 가지 조건으로 관리된다. 누가 읽는지, 어떤 표와 공문 구조가 필요한지, 어떤 계획과 지침을 따라야 하는지, 마지막에 어떤 파일로 제출되는지가 같이 정해져야 한다.

전환: 오늘 예시는 이 조건이 잘 보이는 시스템 점검 안내문으로 잡는다.

5. 첫상대는 누구로 할 것인가?

사용 시간: 2분

설명: 제품화 첫 대상은 반복되고 필수 항목이 분명한 문서가 좋다. 오늘은 내부 업무시스템 점검 안내문을 사용한다. 일정, 영향 서비스, 사용자 조치, 문의처가 모두 구조화하기 좋고, 뒤에서 근거 문서로도 나눌 수 있다.

전환: 먼저 아무 구조 없이 ChatGPT에 맡겼을 때를 본다.

6. 시연 1: 챗지피티에 바로 맡기면

사용 시간: 8분

설명: 이 시연의 핵심은 ChatGPT가 못 쓴다는 것이 아니다. 결과가 자연스러워도 값의 출처, 승인된 표 구조, 검증 규칙이 분리되어 있지 않다는 점을 확인한다.

진행:

1. ChatGPT에 짧은 요청을 붙여 넣는다.
2. 제목, 표, 문의처, 사용자 조치를 확인한다.
3. 표가 있어도 그 표가 승인된 구조인지, 값이 어디서 왔는지 알 수 없다는 점을 짚는다.

사용 문장:

> 내부 업무시스템이 6월 3일 19시부터 22시까지 점검으로 중단됩니다.
> 전자결재와 문서 열람이 제한됩니다.
> 전 직원에게 안내하는 공문을 작성해줘.

전환: 이제 같은 요청에 문서 조건을 붙이면 무엇이 달라지는지 본다.

7. 문서 속 숨은 <표준 주문서> docu meta

사용 시간: 4분

설명: 자동화는 문서 원문에서 바로 시작하지 않는다. 먼저 제목, 수신, 일정, 영향 서비스, 사용자 조치, 문의처처럼 반복되는 자리를 태깅한다. 그 태깅 결과가 필수 슬롯, 표 구조, 근거, 검증 규칙, 출력 형식으로 정리되면 DocuMeta가 된다.

전환: DocuMeta가 실제로 무엇을 고정하는지 본다.

8. docu meta는 무엇을 고정하는가

사용 시간: 5분

설명: DocuMeta는 프롬프트 문장이 아니라 시스템이 읽는 문서 조건이다. 필수 슬롯, 근거, 검증 규칙, 출력 형식을 먼저 고정한다. 처음에는 YAML이나 Markdown으로 시작할 수 있고, 제품 단계에서는 DB와 관리자 화면으로 확장할 수 있다.

유사 개념과 비교: DocuMeta는 완전히 새로운 기술 이름이라기보다, 문서 자동화에서 필요한 여러 개념을 문서 단위로 묶어 부르는 실무적 이름이다. JSON Schema는 데이터 구조와 타입을 검증하지만, 공문 문맥의 근거, 표 계약, 출력 대상까지 설명하지는 않는다. 템플릿 엔진은 빈칸에 값을 넣지만, 그 값이 어디서 왔고 출력해도 되는지 판단하지 않는다. CMS나 폼 빌더는 입력 화면과 저장 구조를 제공하지만, RAG 근거 연결과 검증 판정을 문서 생성 흐름에 자동으로 붙이지는 않는다. 전자결재 양식은 조직 안의 승인된 형식을 고정하지만, LLM이 채울 슬롯, 근거 chunk, 출력 어댑터까지 기계가 읽게 만드는 층은 별도로 필요하다. 그래서 이 강의에서 DocuMeta는 문서 스키마 + 템플릿 계약 + 근거 정책 + 검증 규칙 + 출력 계약을 함께 담는 중간 표현으로 설명한다.

전환: 이제 짧은 DocuMeta만 붙였을 때 결과가 어떻게 달라지는지 본다.

9. 시연 2: 짧은 docu meta만 붙여도

사용 시간: 8분

설명: DocuMeta를 붙이면 좋아지는 것은 문장력이 아니라 통제 가능성이다. 필수 항목과 표 구조가 먼저 고정되고, 값의 출처를 나중에 연결할 자리도 생긴다.

진행:

1. 같은 요청 아래에 짧은 DocuMeta를 붙여 넣는다.
2. 점검 정보 표와 필수 슬롯이 반영됐는지 확인한다.
3. 문의처, 영향 서비스, 사용자 조치가 구조적으로 고정되는 점을 비교한다.

전환: 여기까지가 문서 조건이다. 이제 그 조건에 근거 문서를 붙이는 RAG를 본다.

10. AI 글쓰기의 <패트로누스> RAG

사용 시간: 7분

설명: RAG는 답변을 대신 쓰는 마법이 아니라 근거를 가져오는 층이다. retrieval은 작업계획서, 운영지침, 이전 안내문을 찾고, augmented 단계에서 그 근거를 생성 맥락에 붙이고, generation 단계에서 근거를 바탕으로 값을 만든다. 오늘 시연에서 RAG는 가드레일이 설 수 있게 받쳐주는 스캐폴딩이다.

전환: RAG를 벡터DB와 같은 말로 오해하지 않도록 정리한다.

11. RAG에 대한 흔한 오해

사용 시간: 7분

설명: 벡터DB는 저장소 후보이고, RAG는 검색된 근거를 생성 흐름에 붙이는 방식이다. RAG가 환각을 완전히 없애지는 않지만, 근거를 붙여 환각을 줄이고 검증할 수 있게 만든다. 문서 품질은 RAG 하나가 아니라 문서 조건, 근거 품질, 검증 규칙이 함께 있을 때 안정된다.

전환: 1부를 문서 조건, DocuMeta, RAG의 관계로 묶는다.

12. 1부 요약

사용 시간: 3분

설명: 공문서는 문장만으로 완성되지 않는다. 문서의 반복 구조를 태깅하면 DocuMeta가 되고, DocuMeta는 문서 조건과 출력 기준을 고정한다. RAG는 그 기준 위에 필요한 근거를 찾아 붙인다.

전환: 이제 개념을 실제 파일과 출력 흐름으로 넘긴다.

13. 2부: 배운 규칙이 흐름이 되는 시간

사용 시간: 3분

설명: 2부에서는 방금 본 조건과 근거가 실제 제품 흐름이 되는 장면을 본다. 작업 경로, DocuMeta 프레임, RAG 슬롯 완성, 검증, 출력 어댑터가 순서대로 연결된다.

전환: 먼저 전체 경로를 지도처럼 본다.

14. AI 디지털 글쓰기의 <마법사의 지도>

사용 시간: 5분

설명: 완성본만 보면 어떤 조건과 근거를 통과했는지 알기 어렵다. 제품에서는 조건, 근거, DocuMeta, 검증, 출력이 추적되어야 한다. 이 추적 가능성이 뒤에서 `slot_source_map`, `output_status`, 출력

로그로 나타난다.

전환: 이제 시연 3에서 DocuMeta 프레임을 만든다.

15. 시연 3: DocuMeta 프레임 생성

사용 시간: 15분

설명: 시연 3은 완성 문서를 보여주는 단계가 아니다. 비정형 샘플 원문에서 LLM이 템플릿 설계 초안을 만들고, 사람이 검토해 시연 4가 사용할 DocuMeta 프레임으로 고정하는 단계다.

진행:

1. `demos/03-documeta-completion/in/sample-official-notice.md`를 열어 비정형 원문을 보여준다.
2. `uv run python demos/03-documeta-completion/generate-template-draft.py`를 실행한다.
3. 실행 로그에서 `LLM 응답` 수신인지 확인한다. fallback이 불리면 생성 테스트 실패로 본다.
4. `out/00-template-draft.md`에서 슬롯 후보, 표 계약 후보, QMD/TYP placeholder 후보를 확인한다.
5. `out/documeta.yaml`에서 사람이 고정한 DocuMeta 프레임을 보여준다.
6. `source_contract`, `generation_guardrails`, `table_contracts`, `hwpx_template.blocks`를 짚는다.
7. `demos/03-documeta-completion/notes.md`의 가드레일 `스캐폴딩`을 열어 어떤 장치가 가드레일인지 확인한다.

설명 포인트:

- LLM 초안은 바로 실행 템플릿이 아니라 검토 대상이다.
- DocuMeta 프레임은 시연 4의 `in/documeta.yaml`과 같은 계약이다.
- 가드레일 스캐폴딩은 한국어/슬롯/placeholder 제약, source contract, generation guardrails, output status 계약으로 구성된다.

전환: 이제 이 프레임을 근거 문서로 채워 실제 출력까지 보낸다.

16. docu meta의 마법 레시피

사용 시간: 5분

설명: DocuMeta는 AI 혼자 만드는 완성본이 아니다. AI가 반복 구조와 슬롯 후보를 먼저 찾고, 사람이 필수와 선택, 근거 정책, 출력 조건을 확정한다. 이 협업 구조 덕분에 다음 시연에서 LLM은 자유롭게 쓰는 것이 아니라 정해진 슬롯 안에서만 값을 채운다.

전환: 시연 4에서는 retriever, LLM, 검증, 출력 코드가 각각 무엇을 하는지 분리해서 본다.

17. 시연 4: RAG와 글쓰기 빌드

사용 시간: 17분

설명: 시연 4에서 LLM은 템플릿을 새로 만들지 않는다. FAISS retriever가 가져온 근거를 읽고 DocuMeta의 빈 슬롯을 채운다. 출력 코드는 완성된 DocuMeta를 QMD/TYP/HWPX 템플릿 계약에 주입해 새 공문을 만든다.

진행:

1. `demos/04-build-writing/notes.md`의 `retriever가 가져온 것`, `LLM이 생산한 것`, 가드레일 스캐폴딩을 먼저 확인한다.
2. `demos/03-documeta-completion/out/reference-docs/`에서 시연 3이 생성한 `work-plan.md`, `impact.md`, `previous.md`, `mail.md`를 보여준다.
3. `uv run python demos/04-build-writing/rag-slot-flow.py`를 실행한다.
4. 실행 로그에서 검색 근거와 완성 슬롯을 확인한다.
5. `retrieved_reference_ids`에 `impact`, `previous`, `work-plan`이 있고 `mail`이 빠졌는지 확인한다.
6. `slot_source_map`에서 `schedule -> work-plan`, `scope -> impact`, `required_actions/contact -> previous`를 확인한다.
7. `output_status`가 `출력 가능`인지 확인한다.
8. `uv run python demos/04-build-writing/output-package.py`를 실행한다.
9. 최종 QMD/HTML/PDF/HWPX 결과를 확인한다.

설명 포인트:

- retriever는 글을 쓰지 않고 문서를 고른다.
- LLM은 근거와 DocuMeta 프레임을 함께 보고 빈 슬롯을 채운다.
- 검증/감사 매핑은 어떤 값이 어떤 근거 chunk에서 왔는지 연결한다.
- 출력 코드는 복사 붙여넣기가 아니라 DocuMeta 계약으로 공문 구조를 조립한다.
- 최종 문서에는 근거 문서에 없던 수신/경유/제목 표, 공문 인사말, 점검 표, 붙임, 발신자, 하단 메타데이터 표가 생성된다.
- `04524`처럼 앞자리 0이 있는 값은 문자열로 유지되어야 한다.

전환: 방금 생성된 하나의 DocuMeta가 여러 출력으로 갈라지는 장면을 정리한다.

18. 하나의 주문서, 세 가지 출력

사용 시간: 8분

설명: 같은 DocuMeta가 Quarto, Typst, HWPX로 이어진다. Quarto는 HTML 검토본, Typst는 PDF 조판본, HWPX는 제출 파일 후보로 역할이 다르다. 중요한 것은 세 출력이 서로 다른 프롬프트 결과가 아니라 같은 문서 조건과 표 계약에서 나온다는 점이다.

전환: 이제 도구 이름이 아니라 역할을 기준으로 오픈소스를 본다.

19. 도구선택도 역할 구분 하기 나름

사용 시간: 3분

설명: 오픈소스 이름부터 고르면 제품이 도구에 끌려간다. 먼저 조건 관리, 검색, 생성, 검증, 출력의 역할을 나누어야 한다. 각 역할이 분리되어야 교체도 가능하고 검증도 가능하다.

전환: 역할별로 어떤 오픈소스 후보가 있는지 본다.

20. 역할별 오픈소스 후보

사용 시간: 3분

설명: 도구는 역할별 후보로 본다. 조건 관리는 YAML, JSON Schema, Baserow, NocoDB가 될 수 있고, RAG 연결은 FAISS, LangChain, LangGraph, pgvector가 될 수 있다. 출력은 Quarto, Typst, Pandoc, HWPX 어댑터처럼 문서 형식에 맞춰 선택한다. 여기서 중요한 것은 전부 설치하는 것이 아니라 역할을 분리해 선택하는 것이다.

전환: 이 역할들이 제품 안에서 어떻게 묶이는지 마지막으로 정리한다.

21. 오픈소스로 만드는 AI 디지털 글쓰기 시스템

사용 시간: 2분

설명: 제품은 한 도구가 아니라 연결된 컴포넌트의 조합이다. 작성 화면, DocuMeta, RAG, LLM, 검증 게이트, 출력 어댑터가 연결되어야 한다. 오늘 시연은 이 중 한 문서가 조건에서 출발해 최종 파일까지 통과하는 축소판이다.

전환: 마지막 메시지는 문장이 아니라 통과한 문서다.

22. 마지막에 남는 것은 통과한 문서

사용 시간: 2분

설명: AI 디지털 글쓰기 제품의 결과는 예쁜 문장이 아니다. 문서 조건, 근거, 검증, 출력 계약을 통과한 문서다. 제품화는 여러 도구를 나열하는 데서 시작하지 않고, 문서 하나를 끝까지 통과시키는 데서 시작한다.

마무리: 오늘 본 흐름은 문서 조건 -> DocuMeta -> RAG 근거 -> LLM 슬롯 완성 -> 검증 -> QMD/TYP/HWPX 출력이다.